

Text-to-Mapper: Generating Executable MyBatis ORM Artifacts from Natural Language Requirements

Zhenhan Chen, Xiaoyu Lin, Yingxin Chen, Ruoyu Huang, Jiqiang Yan, Zhenghao Chi, Jiahao Dai, Zhiling Cai, Lijin Wang, and Yiwen Zhong

Abstract—Enterprise database applications are typically mediated by Object-Relational Mapping (ORM) frameworks such as MyBatis rather than by raw SQL, yet most natural-language interfaces to data target single SQL strings and bypass the mapper layer. This paper formulates the pioneering task of *Text-to-Mapper*, generating executable MyBatis mapper files from natural-language requirements given database schemas and Java entity specifications. To support this setting, a high-quality dataset, *MyBatis-Spider* together with an evaluation pipeline that exercises candidates through a parse-build-execute-align runtime chain. To capture execution-level semantics, an execution-based metric, *Object Semantic Alignment* (OSA), compares the Java objects produced by generated and reference mappers. Our empirical study of direct-transfer baselines highlights recurring field-column-type binding errors; motivated by this, we introduce *Across-Struct-Table* (AST), a compact two-dimensional representation that makes these correspondences explicit. Systematic experiments with commercial and open-weights large language models show that AST yields consistent OSA improvements by reducing field-level mismatches. With parameter-efficient finetuning (PEFT) on top of AST, mid-scale open-weights models narrow the gap to larger general-purpose models, and in some settings match or slightly exceed them on this benchmark, suggesting a viable path toward controllable, on-premise Text-to-Mapper systems.

Index Terms—Text-to-Mapper, MyBatis, ORM, code generation, large language models, object semantic alignment, database applications

I. INTRODUCTION

IN modern database-driven software development, ORM frameworks such as MyBatis are pervasive mediators between application code and relational databases. Rather than issuing raw SQL, developers configure mapper interfaces and executable mapper.xml files that encode SQL fragments,

runtime control flow, and result-set mappings into Java objects. A single mapper statement must simultaneously describe the SQL query, branch conditions over parameters (`<if>`, `<choose>`, `<foreach>`), and the alignment between projected columns and the fields and types of target JavaBeans. Seemingly local changes—adding a predicate, renaming a column, or refactoring a nested association—may silently break these cross-artifact contracts and cause production failures, making mapper configuration both time-consuming and error-prone.

Existing industrial tools only partially address this difficulty. Template-based frameworks and code generators (e.g., MyBatis Generator, MyBatis Plus) reverse-engineer schemas to emit Plain Old Java Objects (POJOs), mapper interfaces, and create-read-update-delete (CRUD) XML from templates. These model-first approaches reduce boilerplate and improve static safety, but they presuppose curated schemas and specifications rather than natural-language requirements, and the generated artifacts largely mirror the physical schema. They neither infer business semantics—such as which tables to join under which conditions—nor automatically materialize branch-sensitive control flow and result-set mappings for complex statements. In practice, developers still need to interpret informal requirements, decide which entities and relations are involved, and manually encode cross-artifact contracts in mapper configurations.

Classic natural-language interfaces to databases (NLIDB) and Text-to-SQL systems offer a more semantic alternative: given a schema and a natural-language utterance, they generate an executable SQL string that a client can submit to the Database Management System (DBMS). Recent progress in this area is largely driven by large language models (LLMs), which can jointly consume natural-language descriptions and schema information and learn to align mentions in the requirement with tables, columns, and simple query structures. This suggests that LLM-based generation might also help bridge the gap between informal requirements and mapper configurations. Yet pushing this paradigm from single SQL strings to full MyBatis mapper configurations is non-trivial. A mapper statement intertwines the SQL fragment executed by the DBMS, XML elements that encode branch-sensitive control flow and parameter binding, and Java-side result mappings that reconstruct nested objects. Small inconsistencies across these layers—such as a renamed column not reflected

Manuscript received June 21, 2025; revised [Date]. This work was supported in part by [Your Grant Information]. (Corresponding author: Xiaoyu Lin.)

Zhenhan Chen, Xiaoyu Lin, Yingxin Chen, Ruoyu Huang, Jiqiang Yan, Zhenghao Chi, Jiahao Dai, Zhiling Cai, Lijin Wang, and Yiwen Zhong are with the College of Computer and Information Sciences, Fujian Agriculture and Forestry University, Fuzhou, Fujian 350002, China, and also with the Fujian Provincial Key Laboratory of Smart Agriculture and Forestry, Fujian Agriculture and Forestry University, Fuzhou, Fujian 350002, China. (e-mail: zhenhanchen@fafu.edu.cn; xiaoyulin@fafu.edu.cn; 52311049015@fafu.edu.cn; 718216726@qq.com; yjq@fafu.edu.cn; chizhenghao@fafu.edu.cn; jiahaodai@fafu.edu.cn; zhilingcai@126.com; lijnwang@fafu.edu.cn; yiwzhong@fafu.edu.cn).

This work was financially supported by Natural Science Foundation of Fujian province, China (2023J01078).

0000-0000/00\$00.00 © 2021 IEEE

in the result mapping—can still produce semantically wrong Java objects even when the SQL executes successfully. This motivates automation that can generate *executable* mapper artifacts directly from natural-language requirements, rather than just SQL strings.

In this work, we formulate the *Text-to-Mapper* task: given a natural-language requirement and a database schema, generate an executable MyBatis mapper statement that satisfies ORM-level constraints across SQL, XML control flow, parameter bindings, resultMaps, and associated JavaBeans. Compared with Text-to-SQL, Text-to-Mapper is more demanding because a system must maintain consistency across these artifacts, realize optional filters and nested collections through appropriate control-flow constructs, and ultimately match the behavior of the mapper as seen by the application rather than just its surface form.

These challenges raise two questions: what additional representations and evaluation signals are needed to reliably generate executable mappers, and to what extent existing Text-to-SQL prompting practices transfer to this setting? To study these questions, we construct *MyBatis-Spider*, an executable benchmark for single-statement Text-to-Mapper generation, where each instance couples a natural-language requirement with a concrete schema and a manually verified reference mapper, enabling end-to-end compilation and execution. On top of this benchmark, we define *Object Semantic Alignment* (OSA), a behavior-level metric that compares the Java objects returned by a candidate mapper and a reference mapper.

Using this executable benchmark, we next ask how far existing Text-to-SQL practices carry over. We systematically adapt mainstream Text-to-SQL prompting patterns and schema renderings to the Text-to-Mapper setting, varying task descriptions, schema presentations, and lightweight cross-schema alignment cues. Across models and configurations, we observe that most failures stem not from malformed XML or non-executable SQL, but from *misaligned object semantics*—incorrect or incomplete mappings between result sets and JavaBeans. Simple alignment cues offer modest locality gains but do not resolve field/column/type inconsistencies, motivating inputs that make such correspondences explicit at the statement level.

Building on these observations, we adopt a fixed, domain-constrained prompting backbone and introduce *Across-Struct-Table* (AST), a compact prompt-level representation that surfaces correspondences between database tables and Java entities. AST exposes field-column-type relationships directly in the input, so that branching SQL logic and mapper-level mappings become first-class targets for the model. We instantiate AST-based inputs with both proprietary and open-weights LLM backbones, and further explore parameter-efficient adaptation of medium-scale models. Our results show that naive transfer of Text-to-SQL prompting yields limited gains in behavior-level metrics such as OSA due to persistent object-mapping errors, whereas AST-based inputs consistently improve execution-level correctness; moreover, lightweight adaptation on open-source models narrows or closes the gap to larger proprietary models while maintaining competitive surface and structural behavior.

Contributions:

- **Task & formulation:** We define the *Text-to-Mapper* task: given natural-language requirements and a database schema, generate executable MyBatis mapper artifacts that satisfy ORM-level constraints—runtime control flow, parameter binding, and local consistency between SQL results, resultMaps, and JavaBeans.
- **Executable benchmark & evaluation stack:**¹² We construct *MyBatis-Spider*, an executable benchmark for Text-to-Mapper, together with an automated harness that runs generated mappers end-to-end and defines OSA as an execution-level correctness signal.
- **Method.** We introduce *AST*, a prompt-level structured representation that augments a domain-constrained prompting backbone with explicit field-column-type correspondences and statement-level parameter bindings, and demonstrate that this design transfers across both proprietary and open-source LLM backbones, including parameter-efficient adaptations suitable for private deployment.
- **Evidence:** We present a systematic study of direct Text-to-SQL-style prompting and AST-based inputs across multiple models. The results quantify the limitations of existing practices in the Text-to-Mapper setting and demonstrate that AST, combined with lightweight adaptation, improves OSA while maintaining competitive surface and structural behavior.

We discuss related work on NLIDB/Text-to-SQL, industrial ORM tools, and evaluation metrics in Section II. Section III formalizes the Text-to-Mapper task, the parse-build-execute-align evaluation chain, and introduces the MyBatis-Spider dataset and evaluation metrics. Section IV presents baseline prompting variants transferred from Text-to-SQL and an empirical study of direct transfer. Section V then presents the AST input design and parameter-efficient adaptation. Finally, Section VI reports systematic experiments and ablation studies, followed by limitations, threats to validity, and concluding remarks.

II. RELATED WORK

A. Natural-Language Interfaces to Databases (Text-to-SQL): Scope and Transfer Limitations

Early NLIDB established shared tasks and corpora for mapping text to executable queries (e.g., ATIS, Scholar, Gen-Query) [1]–[4]. With large cross-domain benchmarks (e.g., WikiSQL, Spider) and enterprise-style variants that better approximate business schemas and multi-query workflows (e.g., BIRD, Spider 2.0) [5]–[8], Text-to-SQL has been framed primarily as generalization to unseen schemas. In the LLM era, studies examine zero/few-shot prompting and analyze failure modes [9]–[11]. Beyond prompting, mainstream directions include schema linking and structure-aware encoders (e.g., RAT-SQL, IRNet, LGESQL) [12]–[14], syntax/constraint-aware decoding (e.g., grammar-constrained decoding, PICARD) [15],

¹The MyBatis-Spider dataset is publicly available at: <https://github.com/ghuorahgaorhga/Text-to-Mapper-Dataset>.

²Our evaluation paradigm is open-source and can be accessed at: <https://github.com/ghuorahgaorhga/Text-to-Mapper-Test-Utils>.

[16], execution-guided decoding [17], and curated example selection / in-context few-shot design [18], [19].

Prior Text-to-SQL systems demonstrate that natural-language inputs can be translated into syntactically valid SQL queries. In many applications, however, database access is mediated by ORM frameworks rather than by direct SQL, which adds runtime control-flow, typing, and cross-artifact consistency constraints that are outside the scope of standard Text-to-SQL pipelines. This gap between query generation and executable ORM artifacts remains largely unexplored.

B. ORM Generators and Typed Queries: Capabilities and Limitations

Industrial and academic practice largely adopts model-first or template-based generation. Tools reverse-engineer database schemas—called *model discovery* in Model-Driven Engineering (MDE) [20]—to emit POJOs, mapper interfaces, and CRUD mapper XML (e.g., MyBatis Generator, MyBatis-Plus) [21], [22], while MDE and domain-specific language (DSL) pipelines (e.g., UML→JPA transformations such as MD-JPA, WebDSL, and model-driven API generators) produce persistence code from formal models or specifications [23]–[25]. Typed-query styles similarly reduce boilerplate and improve static safety, but all of these approaches presuppose curated schemas or specifications rather than natural-language requirements, so the generated artifacts mainly mirror the physical database structure. As a result, business semantics remain under-specified: they do not synthesize domain-specific runtime control flow in executable XML or derive join-aware result-set mappings (alias disambiguation, key-based grouping/de-duplication, association/collection in the resultMap) needed to recover a consistent object graph. Prior reports also note architectural side effects such as *anemic domain models* [26] and optional *Active Record* styles [27] that couple persistence to domain entities and can hinder separation of concerns, maintainability, and testing.

C. Evaluation Metrics in Prior Work: Surface, Structural, and Execution Signals

Textual overlap metrics such as BLEU and ROUGE-L are common in database/code generation [28], [29], but they correlate weakly with functional correctness and can reward superficial overlap. For XML artifacts, a lightweight structure-only proxy is to represent each document as the set of absolute XPath-like paths and compare these sets via Jaccard similarity; this proxy, denoted *XPI* for brevity, follows classic XML similarity and clustering ideas (e.g., XRep) [30], [31]. Heavier alternatives include XML-aware tree edit distance [32], the *pq*-gram distance [33], and diff families such as X-Diff [34]. These structural metrics capture hierarchical coverage and element/attribute presence, but they do not attest to executable behavior, parameterized control flow, or mapping correctness between the SQL fragment and resultMap/JavaBeans.

Text-to-SQL evaluations typically pair Exact Set Match accuracy and Execution Accuracy [6]; both execute a single SQL string against a schema and database state and thus do not

translate to artifacts that run through a mapper layer with dynamic control flow, parameter binding, and mapping coverage. Code-oriented overlap metrics are also used in the literature. Among them, CodeBLEU [35] augments BLEU with abstract-syntax-tree-based, keyword, and data-flow terms, instantiated for mainstream programming languages such as Python, Java, JavaScript, and C++. Mapper XML, however, does not have such language-specific frontends, so the CodeBLEU score effectively reduces to n-gram overlap and offers only limited additional signal beyond BLEU/ROUGE-L.

III. TASK DEFINITION, DATASET, AND METRICS

A. Task and Pipeline

Text-to-Mapper is defined as follows. Given a natural-language requirement r , a database schema S , and JavaBean specifications B , the objective is to produce an *executable* MyBatis mapper configuration $\hat{M}$ that realizes r over S and whose result objects conform to B .

A prompt is formed by combining a task description with an explicit rendering of the input artifacts so that structural cues (tables, columns, types, bean fields) are visible to the model; an overview of the workflow is shown in Fig. 1. Formally,

$$R = \text{Render}(S, B) \quad (1)$$

$$P = D(r, R) \quad (2)$$

$$\hat{M} = G(P) \quad (3)$$

where R is a rendering of (S, B) , $D(r, R)$ constructs a task description from r and R , and G denotes the model that generates a candidate mapper $\hat{M}$. In this work, G is instantiated as a LLM, allowing the generator to jointly reason over free-form requirements and rendered schema/bean structure.

To determine executability and localize failures, both $\hat{M}$ and a reference mapper are evaluated by a four-stage runtime chain: *parse* → *build* → *execute* → *align*. For each dataset instance, we record either a successful run or a first-error label $c \in \{\text{Syntax, Build, Execute, Result}\}$, corresponding to a syntax error, build error, execution error, or result-mismatch error, respectively. We use these four names as categorical labels in figures and tables, and dataset-level correctness is aggregated using the execution-based measure OSA, which is defined in detail in Section III together with the precise semantics of each stage in the chain.

B. Boundary to Text-to-SQL

Text-to-Mapper goes beyond emitting a single SQL statement. Text-to-SQL systems typically take a natural-language utterance, a schema, and a database state, and return one query string whose execution result is evaluated in isolation. In contrast, mapper configurations must encode dynamic control flow, parameter binding, and result-set mappings so that the framework can reconstruct Java objects from query results. This gap with Text-to-SQL is best made precise through a real case.

Figure 2 shows an instance from *MyBatis-Spider* (id=6593) drawn from an advertising-agencies schema. The requirement

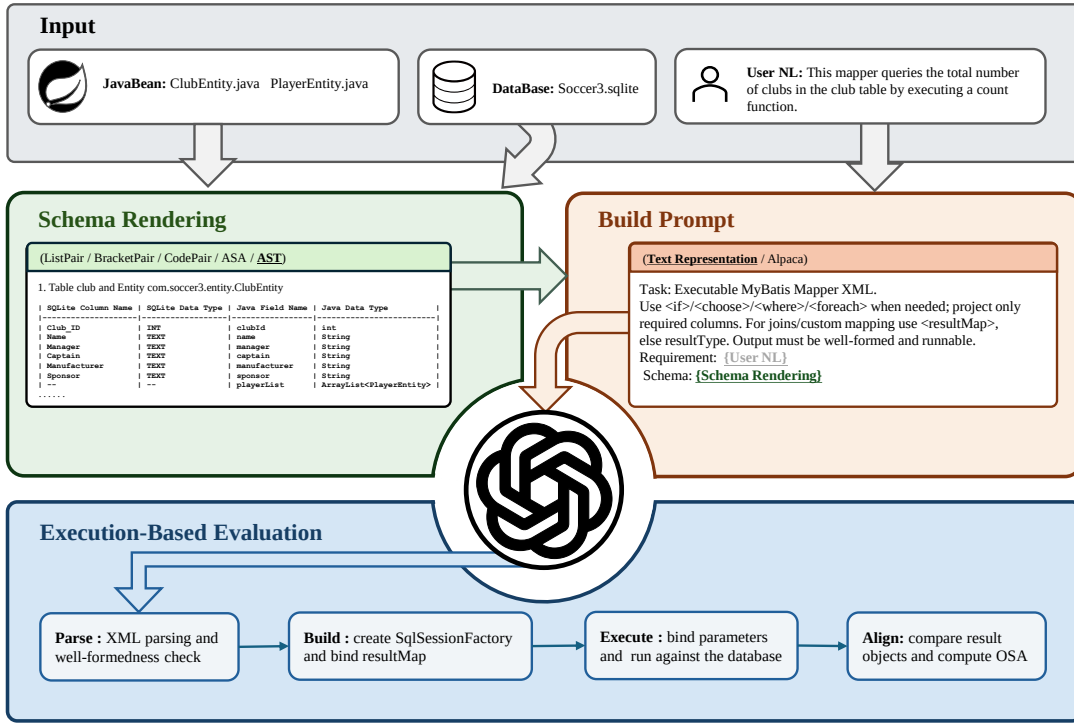


Fig. 1. Workflow overview with a concrete dataset instance (schema: Soccer3.sqlite; entities: ClubEntity, PlayerEntity; requirement: “count clubs”). The bottom panel summarizes the four-stage execution-based check (parse, build, execute, align) used throughout the study.

describes a mapper that fetches a batch of clients and all associated meetings based on a list of client identifiers, and supports dynamic sorting driven by a string parameter. Concretely, the caller passes a map containing clientIds (list of integers, required) and sortBy (string, optional, values “sicCode” or “details”, defaulting to client id). The mapper returns a list of ClientsEntity objects, each populated with a corresponding list of MeetingsEntity objects. The bottom panel of Fig. 2 shows the gold mapper.xml fragment implementing this requirement, including a `<resultMap>` ClientWithMeetingsResultMap and a `<select>` query that joins Clients and Meetings.

For this single requirement, three properties illustrate where Text-to-SQL abstractions stop.

First, the mapper must realize runtime control flow. The `<select>` body combines a `<foreach>` over clientIds, which expands the IN clause by injecting the loop variable id via `#{id}` so that the number of placeholders is determined at runtime, and a `<choose>/<when>/<otherwise>` block that emits different ORDER BY clauses depending on sortBy: ordering by c.sic_code, c.client_details, or c.client_id, all with a secondary order on meeting start time. The effective SQL is therefore a family of queries parameterized by runtime signals, not a single fixed query string.

Second, the mapper must enforce a cross-artifact object-SQL contract. The `<select>` projects columns from the Clients table (aliased c) and the Meetings table (aliased m), while the application expects a hierarchical object graph: each ClientsEntity has scalar fields such as clientId, sicCode, and clientDetails, and a collection property meetingsList of MeetingsEntity instances. The `<resultMap>` ClientWithMeetingsResultMap expresses this contract by binding, for example, `c.client_id ↔ client_id`

`↔ clientId` and `c.sic_code ↔ sic_code ↔ sicCode`, and by declaring a `<collection>` with a columnPrefix so that meeting-related columns are aggregated into each client’s meetingsList. Correctness thus depends on consistent bindings across the SQL fragment, the `<resultMap>`, and the fields and types of ClientsEntity and MeetingsEntity—relationships that single-query Text-to-SQL pipelines neither model nor verify.

Third, the mapper must maintain framework-level consistency within the generated configuration. The `<select>` declares parameterType=“map”, which assumes that the caller provides a map whose keys clientIds and sortBy match the identifiers used in the `<foreach>` and `<when>` tests; a mismatch surfaces when parameters are bound and the statement is executed. The attribute resultMap=“ClientWithMeetingsResultMap” must refer to a `<resultMap>` that is visible in the MyBatis configuration; misspellings or dangling references cause failures when the configuration is parsed or built. These framework-level consistency requirements—across parameter-Type, resultMap, and parameter names—are not checked by exact-set match or execution accuracy on a bare SQL string.

In our setting, a candidate mapper is therefore evaluated as a behavioral object that must survive the full parse→build→execute→align chain, whereas Text-to-SQL metrics operate at the level of a single query string.

C. Dataset Foundations and Source Corpus

Constructing a high-quality dataset for generating MyBatis mapper.xml files from natural language is expensive. Each instance must specify executable query logic, dynamic XML control flow, parameter binding, and Java object mappings,



Fig. 2. Real example (id=6593) used for illustration. The figure contains the original mapper description, the paired SQL query, and a snippet of the target *mapper.xml*.

and must be validated against a real database schema. Fully manual authoring therefore requires annotators with expertise in relational databases, Java, and ORM frameworks, making large-scale curation prohibitive.

To reduce this cost while preserving quality, we start from existing Text-to-SQL corpora and structurally migrate them into Text-to-Mapper form. Among public Text-to-SQL datasets, WikiSQL [5], ATIS [1], [2], SparC [36], and Spider [6] are widely used benchmarks. Table I qualitatively compares these corpora along three axes that matter for enterprise ORM workloads: query complexity, support for multi-table joins, and cross-domain coverage. Spider offers high query complexity, widespread use of multi-table joins, and diverse schemas across domains, while maintaining moderate database sizes that align with typical business applications. We therefore select Spider as the base Text-to-SQL corpus and, in Section III-D, derive the MyBatis–Spider dataset from it.

TABLE I
QUALITATIVE COMPARISON OF REPRESENTATIVE TEXT-TO-SQL
DATASETS USED AS SOURCE CORPORA.

Dataset	Query complexity	Multi-table join	Cross-domain
WikiSQL [5]	Low	✗	✗
ATIS [1], [2]	Medium	Limited	✗
SparC [36]	Medium–High	✓	✓
Spider [6]	High	✓	✓

D. MyBatis-Spider Dataset

1) **Scope and split:** MyBatis-Spider pairs natural-language requirements with executable MyBatis *mapper.xml* artifacts. The dataset contains 6,799 instances over 206 databases, split

by database into 4,645/748/1,406 for train/dev/test. IDs are stable and interpretable: 1–5653 (excluding 3940) form the core subset derived from a convertible portion of Spider (5,652 instances), and IDs ≥ 5654 form the augmented subset (1,147 instances). The test databases do not overlap with train/dev.

2) **Construction pipeline:** Figure 3 overviews a two-phase process. Both phases share the same database schema as the ground truth and use schema-derived Java entity classes as the target object model for MyBatis mappings; each phase produces candidate *mapper.xml* fragments via LLM generation, followed by execution-based validation and expert curation. Only curated, executable mappers are released.

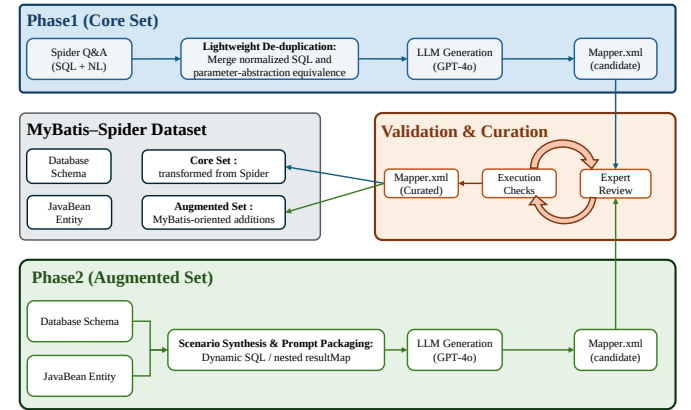


Fig. 3. Two-phase construction of the MyBatis–Spider dataset. Phase 1 converts Spider QA/SQL into de-duplicated *mapper.xml* candidates; Phase 2 synthesizes ORM-oriented cases (dynamic SQL, nested *<resultMap>*, batch via *<foreach>*). Candidates undergo execution checks and expert curation; only curated mappers are released.

Java entity classes (JavaBeans-style). We construct Java entity classes from the SQLite schema using a deterministic, schema-driven generation procedure that mirrors standard schema-to-POJO conventions. We generate one entity class per table, derive class/field names via underscore-splitting and camelCase conversion, and apply deterministic identifier sanitization (e.g., handling Java keyword collisions, leading digits, and special-symbol cases). Importantly, we emit schema-derived Java fields in the same order as the table columns (as returned by PRAGMA table_info), so that the *i*-th Java field corresponds to the *i*-th schema column *by construction*.

To support executable MyBatis artifacts, we additionally generate relationship-typed fields guided by foreign-key constraints and MyBatis *<association>*/*<collection>* conventions for nested object mappings. These relationship/helper fields are appended *after* the schema-derived fields using a deterministic rule, and thus do not affect the positional correspondence of the core schema fields. Finally, we manually validate the generated entities and foreign-key links to repair a small number of hard-to-handle schema issues (e.g., self-referential or malformed foreign keys) and to correct a few remaining low-quality identifiers before incorporating them into the dataset.

Phase 1 (core). Phase I starts from Spider SQL queries, their natural-language questions, and the relevant Java entity classes. It first applies lightweight de-duplication based on normalized SQL and *parameter-abstraction equivalence*,

merging items that reduce to the same parameterized query (e.g., names “Peter”/“Alice” become the same predicate after abstraction). Given each de-duplicated Spider SQL and its question, the system then produces an executable mapper fragment with a matching parameter payload. Each output declares the parameterType; parameters bind one-to-one to the SQL sites and are presented in generic (non hard-coded) form. The parameterType precedence follows common practice: a single simple parameter uses a Java wrapper type (e.g., java.lang.Integer, String); multiple parameters preferentially use an entity class, falling back to map only when no suitable entity exists.

The mapping form is dictated by the SQL: aggregation or single-column projections yield a *resultType-based* `<select>`; multi-column projections or explicit joins yield a *resultMap-based* mapping, introducing `<association>/<collection>` only when the output truly spans tables and requires object hierarchy.

Phase II (augmented). To better reflect ORM workloads, we synthesize additional samples under the target schema. Each instance satisfies at least one of: a resultMap that contains associations/collections/discriminator; *dynamic SQL* (e.g., `<where>/<if>/<foreach>/<choose>`); or *batch retrieval* via `<foreach>` (e.g., an `IN` list). The augmentation prioritizes scenarios that typically require hand-written mappers—nested objects, conditional composition, and batch queries—and sets parameters to be *hitting* (likely non-empty results).

For both phases, a domain expert performs a light revision, and the evaluation platform performs dry-run and live-run checks on the target schema; all released samples pass these checks and are executable.

3) **Coverage and representativeness:** Figure 4 compares four indicators across the core, augmented, and full sets: resultMap usage, dynamic SQL, nested mappings (`<association>`, `<collection>`, `<discriminator>`), and parameter binding. The Spider-derived core under-represents dynamic SQL and nested mappings, whereas the augmented subset explicitly addresses these gaps. Consequently, the full dataset preserves the breadth of SQL logic inherited from Spider while adding substantially improved coverage of MyBatis mapping constructs, yielding more structured mapper artifacts without departing from the original schemas.

E. Evaluation Metrics

We adopt *BLEU-4* [28] and *ROUGE-L* [29] for textual similarity, *XPJ* [30], [31] for structural similarity, and *OSA* for executable semantics (primary). All scores are computed on the same test subset.

a) **BLEU-4:** Let p_n denote clipped n -gram precisions aggregated over the corpus and w_n their weights ($w_n = \frac{1}{4}$ for $n \in \{1, 2, 3, 4\}$). Let r and c be the total reference and candidate lengths. The BLEU score is

$$\text{BLEU} = \text{BP} \cdot \exp\left(\sum_{n=1}^4 w_n \log p_n\right), \quad (4)$$

with brevity penalty

$$\text{BP} = \min(1, e^{1-r/c}). \quad (5)$$

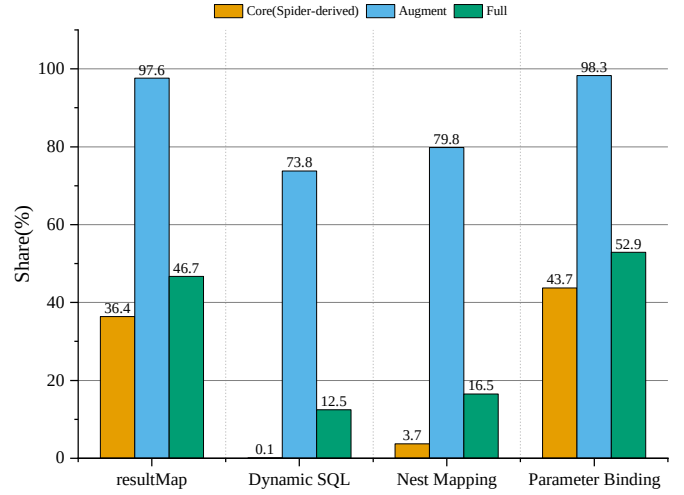


Fig. 4. Coverage of four mapper indicators (resultMap, usage, dynamic SQL, nested mappings, and parameter binding) across the core, augmented, and full sets.

b) **ROUGE-L (F1):** Let $LCS(\hat{M}, M^*)$ be the length of the longest common subsequence between candidate $\hat{M}$ and reference M^* , $|R|$ the length of M^* , and $|C|$ the length of $\hat{M}$. Define

$$R_{LCS} = \frac{LCS}{|R|}, \quad P_{LCS} = \frac{LCS}{|C|}. \quad (6)$$

The F_1 variant is

$$\text{ROUGE-L}_{F_1} = \frac{(1 + \beta^2) P_{LCS} R_{LCS}}{R_{LCS} + \beta^2 P_{LCS}}, \quad \beta = 1. \quad (7)$$

c) **XPJ (XPath-Path Jaccard):** Parsing each mapper into a set of absolute element/attribute paths, $P(\hat{M})$ for the candidate and $P(M^*)$ for the reference, the XPJ score is the Jaccard index

$$\text{XPJ}(\hat{M}, M^*) = \frac{|P(\hat{M}) \cap P(M^*)|}{|P(\hat{M}) \cup P(M^*)|}. \quad (8)$$

Candidates that cannot be safely parsed as XML are assigned $\text{XPJ} = 0$ and included in the average.

d) **OSA (Object Semantic Alignment):** We treat each mapper as a behavioral object evaluated by a four-stage runtime chain: *parse* $\rightarrow$ *build* $\rightarrow$ *execute* $\rightarrow$ *align*. Given a candidate mapper $\hat{M}$ and the reference M^* , both are run under the same database schema and parameter instance.

In the *parse* stage, the mapper XML is parsed into MyBatis configuration objects; XML syntax errors abort the chain and assign the first-error label *Syntax*. In the *build* stage, the framework constructs the `SqlSessionFactory` and registers statements and resultMaps; configuration errors (e.g., unresolved identifiers) terminate the chain with label *Build*. In the *execute* stage, a `SqlSession` is opened, parameters from a test fixture are bound, and the corresponding SQL is executed; runtime exceptions or database access errors terminate the chain with label *Execute*. Only if these three stages all succeed do we enter the *align* stage, where the Java objects returned by $\hat{M}$ and M^* are compared; result mismatches at this stage yield the label *Result*.

Let the two result objects for item i be $\hat{y}_i$ and y_i^* . Alignment rules are: (i) primitive/string values use the host-language equals; (ii) object values are compared recursively over all fields (including nested collections); (iii) collections align position-wise when the SQL contains ORDER BY, and as multisets otherwise. Let $\mathcal{A}(\hat{y}_i, y_i^*) \in \{0, 1\}$ be the item-level alignment predicate, which is 1 if and only if all four stages succeed and the final results align under these rules. Given N items, the OSA score is

$$\text{OSA} = \frac{1}{N} \sum_{i=1}^N \mathcal{A}(\hat{y}_i, y_i^*). \quad (9)$$

Items that fail to parse, build, execute, or align at any stage in the runtime chain are assigned $\mathcal{A} = 0$ and included in the average.

IV. BASELINE METHODS AND DIRECT TRANSFER EXPERIMENTS

A. Naive Strategy

In this section, we build baseline transfer variants by prompting an LLM with (i) a description of natural-language requirements and (ii) a schema/entity rendering (optionally with alignment cues) to generate executable mapper.xml. We evaluate with two requirement templates (a concise text-style template and an instruction-style template, i.e., Alpaca-like) only as a robustness check; unless otherwise noted, we report marginal means averaged over the two templates. Full templates and examples are provided in Appendix A.

1) *Schema renderings*: Following the Text-to-SQL literature, we adopt three database schema formats: column list format [9], bracketed field format [37], and SQL DDL format [38]. We render the JavaBean specifications in parallel and place them adjacent to the database slice, yielding **ListPair**, **BracketPair**, and **CodePair**, which present the SQL schema and the JavaBean schema in the same syntactic family (list/list, bracket/bracket, and code/code; Fig. 5).

2) *Across-Schema Alignment (ASA)*: As an optional ordering cue, ASA places each table block immediately next to its corresponding entity block (“table → entity”), improving locality for long prompts while leaving fine-grained bindings to the model’s inference (Fig. 6).

It is important to distinguish *schema renderings* from the *alignment cue* used later. ListPair/BracketPair/CodePair specify *how* each table and entity is serialized (syntax and granularity of tokens), while Across-Schema Alignment (ASA) specifies *how blocks are ordered* across the two schemas. Applying ASA to a given rendering produces a corresponding *ordering variant* (e.g., *ListPair-Align*), without changing the underlying schema content.

B. Empirical Study of Direct Transfer

We evaluate six baseline variants on the test split ($N=1406$) using GPT-4o: *ListPair*, *ListPair-Align*, *BracketPair*, *BracketPair-Align*, *CodePair*, and *CodePair-Align*, corresponding to the three schema renderings and their ASA-enhanced counterparts. For each generated mapper, we run the

ListPair
Schema_{DB} Table stadium, columns = [Stadium_ID, Location, Name, Capacity, Highest, Lowest, Average] Table singer, columns = [Singer_ID, Name, Country, Song_Name, Song_release_year, Age, Is_male] Schema_{JavaBean} Class com.concertsinger.entity.StadiumEntity, attributes = [stadiumId, location, name, capacity, highest, lowest, average] Class com.concertsinger.entity.SingerEntity, attributes = [singerId, name, country, songName, songReleaseYear, age, isMale]
BracketPair
Schema_{DB} stadium(Stadium_ID, Location, Name, Capacity, Highest, Lowest, Average) singer(Singer_ID, Name, Country, Song_Name, Song_release_year, Age, Is_male) Schema_{JavaBean} Class com.concertsinger.entity.StadiumEntity(stadiumId, location, name, capacity, highest, lowest, average) Class com.concertsinger.entity.SingerEntity(singerId, name, country, songName, songReleaseYear, age, isMale)
CodePair
Schema_{DB} CREATE TABLE "stadium" ("Stadium_ID" int, "Location" text, "Name" text, "Capacity" int, "Highest" int, "Lowest" int, "Average" int, PRIMARY KEY ("Stadium_ID")) Schema_{JavaBean} package com.concertsinger.entity; import java.io.Serializable; import java.util.ArrayList; public class StadiumEntity implements Serializable{ private int stadiumId; private String location; private String name; private int capacity; private int highest; private int lowest; private int average; private ArrayList<ConcertEntity> concertList; }

Fig. 5. Three schema–entity representation combinations for tables and Java entity classes using a slice from the concert_singer database.

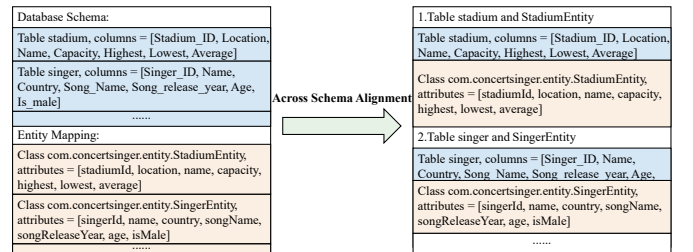


Fig. 6. Across-schema alignment (ASA) applied to a slice from the concert_singer database: each table is followed by its corresponding Java entity.

same four-stage runtime chain *parse*→*build*→*execute*→*align* defined in Section III-A, and apply a first-error short-circuit: as soon as a stage fails, we assign the corresponding error label $c \in \{\text{Syntax, Build, Execute, Result}\}$; if all stages succeed, the run is labeled *Pass*. These per-instance outcome labels support the failure-composition analysis below; dataset-level correctness is still summarized by OSA. Unless otherwise noted, results in this section are reported as *marginal means* (MM) aggregated across the two requirement templates; the per-template breakdown is provided in Appendix B.

1) *Where runs fail first*: Figure 13 presents the first-error composition across the six baseline variants on GPT-4o (MM across the two requirement templates). Each horizontal bar is a 100% stacked histogram over the five outcome labels (Syntax/Build/Execute/Result/Pass); segments are ordered from left to right in runtime-chain order, with the rightmost segment denoting runs that pass all stages (*Pass/OSA*). Thus, the *Pass* segment reflects the share of test instances counted as correct under OSA, and the remaining segments partition failures by the earliest failing stage.

Across all baselines, *Result*-stage failures (i.e., failures in the final *align* stage) account for the largest share of unsuccessful runs. This indicates that object-level mapping semantics—such as *resultMap* wiring, nested *<association>/<collection>*, and *field/column/type* consistency—is a major bottleneck under direct transfer, even when the generated mapper is syntactically valid and can be executed. By contrast, *Syntax-first* cases are rare, and *Build/Execute* errors are comparatively small and stable. Consequently, improvements that do not reduce *Result-first* errors have limited headroom for end-to-end gains under OSA. A per-template breakdown of the same analysis is provided in Appendix B.

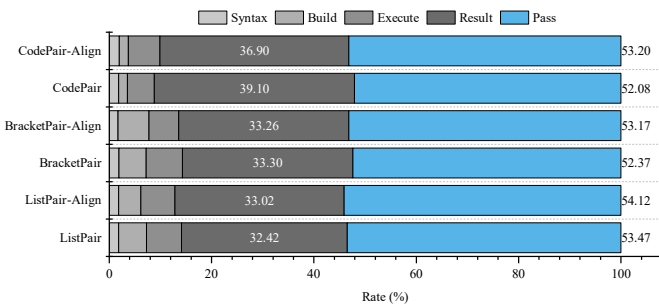


Fig. 7. First-error composition on GPT-4o, reported as marginal means (MM) aggregated across the two requirement templates. Bars are 100% stacked by outcome label (Syntax/Build/Execute/Result/Pass), ordered by runtime stage. The rightmost segment (*Pass/OSA*) denotes runs that successfully complete all stages; the other segments show the share of runs whose *first* failure occurs at each stage. Per-template breakdown is provided in Appendix B.

2) *Effect of lightweight alignment (-Align / ASA)*: We next assess whether a lightweight alignment cue, -Align (ASA), can alleviate these failures by improving the proximity between the relevant table block and its corresponding entity block in the prompt. Figure 8 compares each schema rendering with and without -Align, reporting OSA as MM across the two requirement templates; error bars denote the standard deviation over repeated runs. Across renderings, -Align yields consistent but modest improvements (within roughly one percentage

point), suggesting that proximity alone is insufficient to resolve the dominant *Result*-stage failures observed in Fig. 13. This motivates a stronger alignment mechanism that makes field-column-type bindings explicit and directly usable during mapper generation. Exact gains and additional breakdowns are reported in Appendix B.

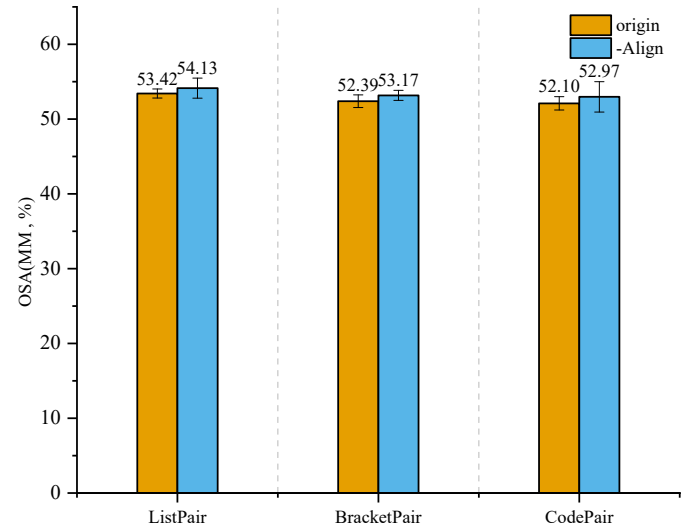


Fig. 8. Effect of a lightweight alignment cue (-Align/ASA) on GPT-4o. Bars report OSA as marginal means (MM) aggregated across the two requirement templates; error bars denote the standard deviation over repeated runs. -Align yields consistent but modest gains across schema renderings, motivating a stronger field-level alignment design (AST).

Taken together, these empirical results suggest that *Result*-stage failures are a primary, recurring source of errors under our evaluated direct-transfer baselines, whereas *Syntax/Build/Execute* errors are comparatively rare and stable. In short, the capacity of naive transfer in our benchmarked setting is often limited by object-level alignment: because these baselines do not provide explicit field-column-type bindings in the prompt, persistent *Result*-first errors at the *align* stage limit end-to-end success under OSA. These observations motivate a field-level alignment design that makes such bindings first-class in the input. The next section introduces *AST*, and Section VI evaluates its impact under the same protocol.

V. ACROSS-STRUCT-TABLE (AST) ALIGNMENT AND PARAMETER-EFFICIENT ADAPTATION

Prior analysis suggests that when database and Java-side information are supplied via simple linear concatenation, the generation process is prone to field-pairing and type-consistency errors, especially when entity classes include auxiliary fields for joins or aggregation. To address these issues at the input level, we introduce *Across-Struct-Table* (AST): a two-dimensional, row-wise alignment that makes candidate correspondences between table columns and Java fields explicit, exposing both names and types in a compact layout. For deployability in private environments, we further adopt a parameter-efficient adaptation strategy to support low-overhead customization. Implementation details and quantitative comparisons are reported in Section VI.

A. AST Representation and Construction

AST converts each table–entity pair into a single four-column grid: [SQLite Column Name | SQLite Type | Java Field | Java Type]. On the database side, column names and types are extracted using the SQLite PRAGMA table_info. On the Java side, field declarations are parsed from the entity class to obtain field names and declared types.

We then align the two sides by position, using the Java fields as row anchors. In *MyBatis-Spider*, entity classes are *schema-derived* and follow a canonical ordering: schema-derived Java fields are emitted in the same order as the table columns (from PRAGMA table_info). Therefore, for the schema-derived portion, the i -th Java field is placed in the i -th row and paired with the i -th schema column name/type. In addition, entity classes may contain relationship/helper fields (e.g., foreign-key induced references) that are appended *after* the schema-derived fields; for these extra fields, the database-side cells are padded with the placeholder “–” so that all rows remain positionally stable.

The grid is rendered as a fixed-width four-column table with a header, a separator line, and aligned rows whose column widths are computed from the actual content. This two-dimensional presentation raises the structural granularity of the input while keeping all original content intact: using the row as the smallest alignment unit exposes name+type pairs explicitly and stably to guide resultMap and SQL generation. The procedural steps coincide with Algorithm 1, and an illustrative slice on public data appears in Fig. 9.

Algorithm 1 Construction of Across-Struct-Table (AST)

- 1: **Input:** SQLite table T_k and corresponding Java class J_k
 - 2: **Output:** structured representation $AST_{T_k} = \langle (c_i^*, d_i^*, f_i^*, e_i^*) \rangle$, used as model input
 - 3: Execute PRAGMA table_info(T_k) to extract schema fields:
 - 4: $C_k = \{(c_i, d_i)\}_{i=1}^n$
 - 5: Parse J_k to extract Java field declarations:
 - 6: $F_k = \{(f_j, e_j)\}_{j=1}^m$
 - 7: Initialize $AST_{T_k} \leftarrow []$
 - 8: **for** $i = 1$ **to** m **do**
 - 9: **if** $i \leq n$ **then**
 - 10: $(c_i^*, d_i^*) \leftarrow (c_i, d_i)$
 - 11: **else**
 - 12: $(c_i^*, d_i^*) \leftarrow (“–”, “–”)$
 - 13: **end if**
 - 14: $(f_i^*, e_i^*) \leftarrow (f_i, e_i)$
 - 15: Append quadruple $(c_i^*, d_i^*, f_i^*, e_i^*)$ to AST_{T_k}
 - 16: **end for**
 - 17: Compute column widths from header labels and values for pretty-printing
 - 18: Format header, separator, and rows using the aligned column widths
-

B. Difference from Naive Concatenation and Rationale

Baseline schema renderings concatenate table definitions and Java entities as separate blocks of text (Section IV-A), optionally reordered by ASA. In this layout, relationships

between columns and fields remain implicit: the model must infer pairings from names and surrounding context. Our first-error analysis in Section IV-B shows that such implicit alignment is a major source of *Result*-stage failures, while *Syntax/Build/Execute* behavior is comparatively stable.

AST instead encodes these cross-structure relations as explicit row-wise alignments. Each Java field anchors a row, and the corresponding column name and type (or the placeholder “–” when no column is present at that position) are placed in the same row. This length equalization preserves row indices even when table slices and entity classes have different cardinalities and ensures that fields or columns appearing on only one side remain visible rather than being elided. Co-locating names and types within a fixed four-column grid gives the model localized cues for field–column–type consistency: generating a resultMap entry or a SELECT list can be guided by same-row lookup rather than by diffuse similarity across long modular blocks. This design is intended to directly target the dominant *Result*-first errors observed under naive concatenation, where mismatched or missing mappings between result sets and JavaBeans limited end-to-end OSA.

C. Role of Parameter-Efficient Adaptation

Although the AST-based rendering of schemas already yields gains for zero-shot Text-to-Mapper generation, we still observe persistent errors on instances with long branch-sensitive logic, deeply nested result mappings, and project-specific naming conventions. From an engineering perspective, it is natural to further specialize a backbone model to the conventions and schemas of a particular codebase. However, many realistic deployments operate under strict resource and compliance constraints: enterprises often hold proprietary and sensitive data and prefer to keep adaptation on-premise, while full-parameter fine-tuning of large models is costly in terms of memory, training time, and storage, and each customized variant effectively becomes a separate model to maintain. Previous work on PEFT shows that updating only a small fraction of parameters can achieve performance comparable to full fine-tuning while drastically reducing computation and storage costs [39]. In privacy-sensitive and federated settings, PEFT has also been highlighted as a natural way to adapt large foundation models without sharing raw data, since only lightweight task-specific deltas need to be stored and communicated [40]. For code-generation workloads in particular, empirical studies report that PEFT techniques can outperform purely prompt-based adaptation (e.g., in-context learning and retrieval-augmented prompting) while remaining resource-efficient across multiple code benchmarks [41].

Motivated by these considerations, we adopt LoRA [42] as our parameter-efficient adaptation mechanism for Text-to-Mapper. The base model remains largely frozen, and small low-rank adapters are inserted into attention-related linear projections. During fine-tuning, only these adapter parameters are updated with the standard autoregressive objective. In our implementation, we reuse exactly the same Text-to-Mapper pipeline and AST-based prompts as at inference time (Section III-A and Section V-A), and fine-tune adapters on

the train split of MyBatis-Spider by pairing AST-formatted prompts with their reference mapper files. Training hyperparameters (such as LoRA rank and scaling, learning rate, and adapter placement) and the empirical impact on backbones like LLaMA3-8B-Ins and GLM4-9B-Chat are reported in Section VI.

Across-Struct-Table			
1. Table stadium and Entity com.concertsinger.entity.StadiumEntity			
SQLite Column Name	SQLite Data Type	Java Field Name	Java Data Type
Stadium_ID	INT	stadiumId	int
Location	TEXT	location	String
Name	TEXT	name	String
Capacity	INT	capacity	int
Highest	INT	highest	int
Lowest	INT	lowest	int
Average	INT	average	int
--	--	concertList	ArrayList<ConcertEntity>
2. Table stadium and Entity com.concertsinger.entity.StadiumEntity			
SQLite Column Name	SQLite Data Type	Java Field Name	Java Data Type
Singer_ID	INT	singerId	int
Name	TEXT	name	String
Country	TEXT	country	String
Song_Name	TEXT	songName	String
Song_release_year	TEXT	songReleaseYear	String
Age	INT	age	int
Is_male	bool	isMale	boolean
--	--	concertList	ArrayList<ConcertEntity>

Fig. 9. Illustration of the AST strategy using a slice from the concert_singer database.

VI. SYSTEMATIC EXPERIMENTS

A. Experimental Setup

All experiments share the same database initialization, Java Database Connectivity (JDBC) type mapping, session management, and parameter binding. We evaluate five representative LLMs covering commercial and open-weights families: GPT-4o [43], GLM4-plus and GLM4-9B-chat [44], LLaMA3-8B-Instruct [45], [46], and DeepSeek-V3.2 [47], [48]. To keep tables concise, we use short labels in tables (e.g., LLaMA3-8B-Ins) while citing official names in prose.

Inputs vary mainly by *schema/entity representations*, including three schema renderings (ListPair, BracketPair, CodePair), their ASA-enhanced counterparts (denoted “-Align”), and the proposed AST representation. We use two requirement templates (text-style and instruction-style) only for robustness and report *marginal means* (MM) aggregated across templates. Each configuration is run with three independent random seeds; tables report the mean $\pm$ standard deviation, and for MM entries we combine per-template standard deviations via a root-mean-square (RMS) aggregation. Detailed per-template results are deferred to Appendix C.

Our principal metric is OSA, reflecting execution-level functional correctness. We use XPJ as the structural-similarity indicator alongside BLEU-4 and ROUGE-L. Decoding uses temperature 0.1, top-p 1.0, and a maximum of 512 new tokens.

We adapt mid-scale open-weights backbones with frozen weights and low-rank adapters on attention projections (q_proj, k_proj, v_proj). Training uses the *MyBatis-Spider* train+dev splits. A single fixed configuration is used throughout: LoRA rank $r=16$, scaling $\alpha=32$, dropout 0.05; context window 2048 and target cap 512; optimization with $lr=1e-4$, $weight_decay=0$,

for 3 epochs and an effective batch size of 1×16 via gradient accumulation (batch=1, accum=16); linear warmup of 5%. We evaluate with the same packaging and decoding protocol as zero-shot runs and summarize results under the same blocked design.

B. Rendering and ASA (Across-Schema Alignment)

We analyze the main effects of schema rendering and the impact of ASA. For each model and configuration among (*ListPair*, *ListPair-Align*, *BracketPair*, *BracketPair-Align*, *CodePair*, *CodePair-Align*), Table II reports the marginal mean OSA before and after applying Across-Schema Alignment (ASA), together with the change (Δ ASA) and the shift in the distribution of first-error labels (Syntax/Build/Execute/Result).

Two main observations arise. First, ASA consistently produces small yet positive OSA gains across most configurations (typically +0.5–1.5 pp). Second, the Δ Stage columns reveal that ASA primarily reduces the frequency of *Build* and *Execute* first-error labels, while the proportion of *Result*-labeled failures (i.e., failures in the final *align* stage) remains high. This pattern suggests that ASA mainly enhances structural stability and parameter binding along the *parse*→*build*→*execute* stages, but cannot fully resolve field-level mapping mismatches at the *align/Result* stage—an issue revisited in Section VI-C.

C. Structured Input: AST—Field-Level Alignment and Its Effects

We now evaluate AST, a prompt-level representation that makes field–column–type correspondences explicit as `bean.field ↔ table.column(alias) ↔ java.type`. AST consolidates heterogeneous schema sources into a positionally stable grid; when the two sides differ in length, placeholders such as – keep row indices stable so that fields appearing on only one side remain visible rather than elided. By exposing the bindings later consumed by the executor (e.g., typing and aliases in `<resultMap>`), AST is designed to address a common end-to-end failure mode in our evaluation: result-stage mismatches arising from misbindings, aliasing, or type conversions.

Table III reports BLEU-4/ROUGE-L/XPJ together with executability (OSA), all as marginal means (MM) across prompt templates. Row-wise maxima (bold) show that AST achieves the best OSA (MM) for all compared backbones. The gains are model-dependent but consistent across runs, aligning with AST’s goal of reducing latent binding errors without changing the backbone or decoding strategy.

To localize where the gains come from, Table IV contrasts AST with the best non-AST configuration per backbone under marginal means across styles. AST improves OSA on all five models by +0.94 to +1.88 pp (mean $\approx$ +1.24 pp). In terms of first-error labels, AST predominantly *reduces* failures labeled *Result* (i.e., failures in the final *align* stage): −4.86 (GLM4-9B-chat), −0.31 (GPT-4o), −1.62 (DeepSeek-V3.2), and −1.03 (GLM4-plus), averaging $\approx$ −1.27 pp. The exception is LLaMA3-8B-Ins, where *Result* increases by +1.46 pp while *Syntax/Build* drop by −2.54/ − 0.64 pp; this pattern suggests

TABLE II
RENDERING AND ASA: OSA AND FIRST-ERROR LABEL DIFFERENCES (MARGINAL MEANS).

Model	Rendering	OSA _{Origin} (MM)	OSA _{Align} (MM)	Δ ASA	Δ Syntax	Δ Build	Δ Execute	Δ Result
LLaMA3-8B-Ins	ListPair	21.43	22.88	+1.45	-0.40	-1.20	-0.80	+2.40
	BracketPair	22.73	23.55	+0.82	-0.30	-0.60	-0.50	+1.40
	CodePair	23.55	24.10	+0.55	-0.10	-0.50	-0.40	+1.00
GLM4-9B-chat	ListPair	10.72	11.37	+0.65	-0.20	-1.10	-0.70	+2.00
	BracketPair	11.26	12.21	+0.95	-0.10	-1.00	-0.80	+1.90
	CodePair	11.09	11.43	+0.34	-0.20	-0.80	-0.60	+1.60
GPT-4o	ListPair	53.42	54.13	+0.71	-0.30	-0.90	-0.50	+1.70
	BracketPair	52.38	53.16	+0.78	-0.10	-0.70	-0.60	+1.40
	CodePair	52.10	53.19	+1.09	-0.20	-0.60	-0.70	+1.50
DeepSeek-V3.2	ListPair	48.41	49.04	+0.63	-0.40	-0.80	-0.70	+1.90
	BracketPair	48.54	48.68	+0.14	-0.20	-0.40	-0.50	+1.10
	CodePair	48.99	49.63	+0.64	-0.10	-0.60	-0.60	+1.30
GLM4-plus	ListPair	42.83	44.28	+1.45	-0.30	-0.90	-0.60	+1.80
	BracketPair	43.95	44.53	+0.58	-0.10	-0.70	-0.60	+1.40
	CodePair	42.30	44.02	+1.72	-0.10	-0.80	-0.70	+1.60

Notes: MM (marginal mean) denotes the average over the two requirement templates (text-style and instruction-style). Δ ASA = OSA_{Align} - OSA_{Origin}. Δ Syntax, Δ Build, Δ Execute, and Δ Result denote the change in the share (percentage points) of test instances whose *first error* is labeled *Syntax*, *Build*, *Execute*, or *Result*, respectively, computed as share_{Align} - share_{Origin}. These labels correspond to the four stages of the runtime chain *parse*→*build*→*execute*→*align*. Positive Δ Stage indicates an increased share under Align.

that AST removes early-stage fragility so that more runs reach object-level comparison, yet residual field-column-type mismatches still surface at the end. Changes in the *Execute* label are small and model-dependent (from -1.01 to +1.04 pp), consistent with AST acting mainly on binding/typing—e.g., aliases and types in `<resultMap>` and correct parameterType usage—rather than on execution control. Overall, probability mass leaving non-execution error labels is largely transferred to OSA, and the average reduction in *Result*-labeled failures is consistent with interpreting AST as a field-level alignment that addresses a frequent bottleneck in our runtime chain.

Overall, AST is a minimally invasive, prompt-level alignment that is intended to address a frequent failure point in our pipeline—field-level mapping—achieving the highest OSA within each backbone’s row while leaving the surface form largely intact. A detailed discussion of why auxiliary text/structure metrics may move differently from OSA appears in Section VI-D.

D. Metric Divergence

Non-execution metrics—BLEU-4, ROUGE-L, and the structure-oriented XPJ—do not adequately evaluate the task of generating executable mapper.xml. Our objective is *runtime* functional correctness, captured by OSA. Evidence from Table III and the case analyses below shows persistent misalignment between text/structure similarity and executable semantics, both *within* a model and *across* models.

a) Within-model divergence.: Even under a fixed backbone and controlled inputs, the configuration that maximizes OSA is often *not* the one that maximizes BLEU/ROUGE/XPJ in the same row block of Table III. To reduce the emphasis on prompt formatting, we report OSA primarily as the marginal mean across templates, OSA (MM). For example, with **GPT-4o**, BLEU-4 and ROUGE-L peak at *ListPair-Align* (27.87 and 65.99), whereas *AST* yields the best executability (OSA (MM) 56.01, compared to the best non-AST baseline OSA

(MM) 54.13). Similarly for **LLaMA3-8B-Ins**, BLEU-4 and XPJ peak at *CodePair-Origin* (BLEU 12.62, XPJ 52.40), while executability is highest under *AST* (OSA (MM) 25.23, compared to the best non-AST baseline OSA (MM) 24.10).

b) Across-model divergence.: Rankings induced by BLEU/ROUGE/XPJ across backbones also diverge from OSA. For instance, **DeepSeek-V3.2** attains the highest XPJ among compared systems (e.g., 77.62 at *ListPair-Align*), yet its best executability (AST, OSA (MM) 50.57) trails **GPT-4o** (AST, OSA (MM) 56.01). Conversely, models leading OSA need not top BLEU/ROUGE/XPJ. Hence, text/structure metrics are neither valid proxies for executable semantics nor reliable tie-breakers when OSA is close.

c) Why similarity metrics diverge from executability.: BLEU/ROUGE reward token overlap and local *n*-grams; XPJ tracks structural edits of XML paths. Mapper files admit semantically equivalent realizations with different join orders, aliases, tag orderings, and identifier choices, which can depress surface similarity despite correct execution. Conversely, binding/typing errors (e.g., missing parameterType, alias mismatches carried into `<resultMap>`, or JDBC conversions) may remain largely invisible to surface metrics and often first manifest at the *Result* stage. Even structure-aware similarity cannot verify behaviors governed by dynamic tags, parameter substitution paths, or collection ordering when SQL lacks explicit ORDER BY.

d) Case studies.: To make the above divergence concrete, we present two representative instances. Figure 10 shows a *false positive* by surface-oriented metrics: the generated `<select>` omits `parameterType="java.lang.String"`, so the placeholder binds as NULL at runtime; the predicate degenerates and the result set is incorrect. Textual and structural similarities are nevertheless high (BLEU-4: 68.01, ROUGE-L: 62.50, XPJ: 80), so BLEU/ROUGE/XPJ would misclassify this example as good, whereas OSA correctly flags a semantic failure (OSA = 0).

TABLE III
CROSS-RENDERING PERFORMANCE WITH AND WITHOUT ALIGNMENT.

Model	Metrics	ListPair		BracketPair		CodePair		AST
		Origin	Align	Origin	Align	Origin	Align	
LLaMA3-8B-Ins	BLEU-4 (MM)	11.71 ± 0.29	11.28 ± 0.06	10.96 ± 0.19	11.47 ± 0.13	12.62 ± 0.07	11.87 ± 0.84	12.17 ± 0.78
	ROUGE-L (MM)	50.91 ± 0.07	51.47 ± 0.20	51.23 ± 0.20	52.08 ± 0.87	52.06 ± 0.28	51.88 ± 1.60	52.55 ± 1.61
	XPJ (MM)	50.51 ± 0.33	48.72 ± 0.12	48.62 ± 0.20	50.33 ± 0.40	52.40 ± 0.34	50.48 ± 2.42	52.17 ± 2.35
	OSA (MM)	21.43 ± 0.35	22.88 ± 0.50	22.74 ± 2.17	23.56 ± 1.11	23.56 ± 0.79	24.10 ± 0.58	25.23 ± 0.47
GLM4-9B-chat	BLEU-4 (MM)	11.95 ± 0.12	12.80 ± 0.18	12.38 ± 0.19	13.77 ± 0.26	12.13 ± 0.17	12.36 ± 0.11	14.28 ± 0.22
	ROUGE-L (MM)	44.60 ± 0.13	46.02 ± 0.22	45.41 ± 1.12	46.90 ± 0.76	45.00 ± 0.42	46.85 ± 0.23	46.89 ± 0.83
	XPJ (MM)	53.70 ± 0.40	53.30 ± 0.39	54.23 ± 0.30	53.47 ± 0.13	49.10 ± 0.28	53.27 ± 0.31	53.33 ± 0.33
	OSA (MM)	10.72 ± 0.61	11.37 ± 0.56	11.26 ± 1.11	12.21 ± 0.26	11.10 ± 0.31	11.43 ± 0.42	13.29 ± 0.10
GPT-4o	BLEU-4 (MM)	25.96 ± 0.47	27.87 ± 1.96	26.29 ± 0.74	27.59 ± 0.93	26.90 ± 0.76	27.31 ± 1.07	26.43 ± 0.78
	ROUGE-L (MM)	64.70 ± 0.26	65.99 ± 0.82	64.72 ± 0.45	65.81 ± 0.54	65.89 ± 0.56	65.66 ± 0.38	65.22 ± 0.41
	XPJ (MM)	70.00 ± 0.60	72.00 ± 1.71	70.51 ± 0.74	71.93 ± 1.19	72.79 ± 1.11	72.00 ± 1.30	70.58 ± 0.76
	OSA (MM)	53.42 ± 0.60	54.13 ± 1.43	52.39 ± 0.90	53.17 ± 0.69	52.10 ± 0.99	53.19 ± 2.95	56.01 ± 0.53
DeepSeek-V3.2	BLEU-4 (MM)	25.99 ± 0.08	26.87 ± 0.20	25.92 ± 0.13	26.32 ± 0.19	24.51 ± 0.21	22.29 ± 0.10	25.07 ± 0.23
	ROUGE-L (MM)	64.43 ± 0.04	65.39 ± 0.19	64.48 ± 0.14	65.20 ± 0.28	65.35 ± 0.13	65.36 ± 0.14	65.77 ± 0.10
	XPJ (MM)	75.99 ± 0.14	77.62 ± 0.38	76.18 ± 0.47	77.22 ± 0.48	77.55 ± 0.32	77.55 ± 0.20	77.41 ± 0.30
	OSA (MM)	48.41 ± 0.52	49.04 ± 0.50	48.54 ± 0.38	48.68 ± 0.48	48.99 ± 0.50	49.64 ± 0.79	50.57 ± 0.23
GLM4-plus	BLEU-4 (MM)	24.23 ± 0.16	23.97 ± 0.94	23.64 ± 0.28	23.45 ± 0.21	21.66 ± 0.27	21.75 ± 0.26	24.34 ± 1.09
	ROUGE-L (MM)	59.17 ± 0.13	59.18 ± 0.62	58.99 ± 0.24	59.05 ± 0.07	58.29 ± 0.21	58.91 ± 0.16	59.44 ± 0.61
	XPJ (MM)	66.90 ± 0.38	67.06 ± 0.83	66.34 ± 0.32	66.33 ± 0.13	66.61 ± 0.19	66.68 ± 0.40	67.27 ± 0.82
	OSA (MM)	42.83 ± 1.12	44.29 ± 1.37	43.96 ± 0.80	44.54 ± 0.61	42.30 ± 0.82	44.02 ± 0.57	45.70 ± 0.73

Notes: “Origin/Align” denote non-aligned and ASA-aligned variants, respectively. All metrics are reported as marginal means (MM) across the two requirement templates (text-style and instruction-style); the corresponding standard deviation is combined via RMS: $\sqrt{(s_{\text{Text}}^2 + s_{\text{Alpaca}}^2)/2}$. All values are mean±std over three runs. Row-wise maxima (by mean) are bolded.

TABLE IV
AST VS. BEST NON-AST PER MODEL (MARGINAL MEANS ACROSS STYLES).

Model	OSA (Best)	OSA (AST)	ΔOSA	ΔSyntax	ΔBuild	ΔExecute	ΔResult
LLaMA3-8B-Ins	24.10	25.23	+1.13	-2.54	-0.64	+0.58	+1.46
GLM4-9B-chat	12.21	13.29	+1.08	+3.36	+1.47	-1.01	-4.86
GPT-4o	54.13	56.01	+1.88	+0.06	-0.39	-0.50	-0.31
DeepSeek-V3.2	49.63	50.57	+0.94	+0.11	-0.02	+0.59	-1.62
GLM4-plus	44.54	45.70	+1.16	+0.19	-1.37	+1.04	-1.03

Best non-AST (per row): LLaMA3-8B-Ins = *CodePair-Align*; GLM4-9B-chat = *BracketPair-Align*; GPT-4o = *ListPair-Align*; DeepSeek-V3.2 = *CodePair-Align*; GLM4-plus = *BracketPair-Align*. ΔOSA = OSA_{AST} − OSA_{Best} (percentage points). ΔSyntax, ΔBuild, ΔExecute, and ΔResult are percentage-point changes in the shares of test instances whose *first error* is labeled *Syntax*, *Build*, *Execute*, or *Result*, respectively, computed as share_{AST} − share_{Best}. These four labels correspond to the stages of the runtime chain *parse*→*build*→*execute*→*align*. Stage shares and OSA are equal-weight marginal means over the two requirement templates. Small mismatches between $\sum \Delta\text{Stage}$ and −ΔOSA are due to rounding.

Figure 11 shows the complementary *false negative* case for surface-oriented text metrics. The generated mapper is functionally equivalent to the reference, but differs in join order, alias choices, top-level tag ordering, and identifier/formatting. These harmless refactorings sharply reduce n-gram overlap (BLEU-4: 1.65, ROUGE-L: 38.92) despite perfect structural overlap (XPJ: 100), so BLEU/ROUGE would incorrectly judge this prediction as poor. In contrast, OSA reports OSA = 1 and correctly reflects that the two mappers induce the same runtime behavior.

These cases exemplify the quantitative patterns above and explain why OSA should remain the primary basis for model selection and conclusions, with similarity metrics used only as auxiliary references.

E. Impact of Adopting Parameter-Efficient Fine-Tuning

To further enhance the structural adaptability and generation accuracy of medium-scale open-source LLMs in the Text-to-Mapper task, we employ LoRA to perform lightweight supervised fine-tuning on two models: LLaMA3-8B-Ins and GLM4-9B-chat. Crucially, the fine-tuning is conducted under the constraint that the backbone model parameters remain frozen. All experiments adopt the *AST* structural representation and the instruction-style template, a strong configuration for these two backbones identified in earlier experiments. All runs are conducted on a single NVIDIA RTX 3090 GPU, ensuring practical portability in typical hardware environments.

Table V reports performance before and after LoRA under *Instruction-style* + *AST*. All evaluation metrics improve *consistently* after fine-tuning, including execution-level OSA



Fig. 10. A false positive case from GLM4-plus BracketPair. The generated mapper achieves high textual similarity scores (BLEU-4: 68.01, ROUGE-L: 62.50, XPJ: 80) but fails functionally (OSA: 0) due to a missing parameter-Type attribute required by the MyBatis framework.



Fig. 11. A false negative case from GPT-4o with AST. The generated mapper is functionally correct (OSA: 1) but receives unfairly low textual similarity scores (BLEU-4: 1.65) due to stylistic differences, such as an alternative but equivalent SQL join order.

as well as surface metrics (BLEU-4, ROUGE-L) and the structure-oriented indicator XPJ. For example, BLEU-4 for **LLaMA3-8B-Ins** improves from 9.24 to 71.84, and for **GLM4-9B-chat** from 14.76 to 72.12.

We term this phenomenon **Holistic Alignment**: supervised fine-tuning steers the model to concurrently master the underlying logic (functional correctness) and the surface conventions (code style) of the task. The synchronized gains suggest that adapters internalize field-column-type regularities exposed by AST while also learning syntactic templates and idiomatic patterns of reference mappers, thereby reinforcing the production of functionally correct <mapper> artifacts alongside improved textual/structural similarity.

TABLE V
IMPACT OF LORA FINE-TUNING ON MODEL PERFORMANCE
(INSTRUCTION-STYLE + AST).

Model	Metric	Before	After	Δ pp
LLaMA3-8B-Ins	BLEU-4	9.24 $\pm$ 0.07	71.84 $\pm$ 0.03	+62.60
	ROUGE-L	50.23 $\pm$ 0.11	89.76 $\pm$ 0.06	+39.53
	XPJ	46.91 $\pm$ 0.14	97.43 $\pm$ 0.02	+50.52
	OSA	23.52 $\pm$ 0.62	72.05 $\pm$ 0.36	+48.53
GLM4-9B-chat	BLEU-4	14.76 $\pm$ 0.23	72.12 $\pm$ 0.24	+57.36
	ROUGE-L	48.58 $\pm$ 1.12	89.71 $\pm$ 0.07	+41.13
	XPJ	55.72 $\pm$ 0.23	97.35 $\pm$ 0.05	+41.63
	OSA	15.86 $\pm$ 0.14	68.59 $\pm$ 1.40	+52.73

F. Prompt length and content vs. structure.

To assess whether AST's gains are mainly driven by longer inputs, Table VI reports the prompt token length for the main renderings under the LLaMA3-8B-Instruct tokenizer. AST is longer than identifier-centric renderings (ListPair/BracketPair) partly because it makes column/field *type cues* explicit. In contrast, compared to CodePair (SQL DDL + Java declarations), AST does not introduce extra schema semantics beyond names and types; its token overhead mainly comes from the grid-style formatting tokens (separators, padding, and alignment symbols) used to enforce row-wise locality. Therefore, improvements of AST over CodePair are unlikely to be explained solely by "more information", but are better attributed to the explicit cross-structure alignment encouraged by the structured layout.

TABLE VI
PROMPT TOKEN LENGTH SUMMARY FOR MAIN SCHEMA RENDERINGS
UNDER THE LLAMA3-8B-INSTRUCT TOKENIZER (AVG./MIN/MAX/P90).

Schema rendering	Avg.	Min	Max	P90
ListPair	303.02	72	2406	591
BracketPair	253.70	52	2160	501.4
CodePair	767.74	177	4878	1679
AST	857.04	205	6901	1816

G. Limitations and Threats to Validity

This study has several limitations and potential threats to validity.

Internal validity. We control major confounders by treating problem-statement style as a blocking factor and reporting marginal means across styles; decoding uses fixed temperature/top- p and three independent seeds per setting. Minor nondeterminism from commercial APIs and implementation details (e.g., JDBC driver versions) may still affect individual items, but the reported results are averaged over repeated runs. We select the “best non-AST” baseline for each backbone as the configuration with the highest OSA marginal mean (the equal-weight average over the two templates), so that AST is compared against a single, fixed reference per model. Since some improvements are small (on the order of 1 pp), they should be interpreted together with the reported standard deviations.

External validity. Our experiments target MyBatis `<mapper>` artifacts on SQLite backends across 206 databases; results may not fully transfer to other ORMs (e.g., JPA/Hibernate), database engines (e.g., MySQL/PostgreSQL with vendor-specific SQL), or production schemas with stored procedures, triggers, or sharding.

Although the execution loop and aligner are portable in principle, adapting them to other technology stacks and deployment settings requires additional engineering, and absolute performance numbers should be interpreted in light of these differences.

Methodological scope. AST is a prompt-level alignment that surfaces `bean.field` $\leftrightarrow$ `table.column(alias)` $\leftrightarrow$ `java.type` and is designed to address the dominant failure mode observed in Section IV-B: field-column-type mismatches that manifest at the Result stage. It does not correct deeper logical errors such as missing join predicates, incorrect aggregation logic, or mis-specified business conditions. In addition, our current instantiation of AST is serialized as a flat token sequence; for larger schemas, this representation can become long relative to typical context windows and may require sharding or more hierarchical variants, which we leave to future work.

Data limitations. Despite execution-time validation and expert curation, residual annotation noise may remain. Domain and type coverage are finite: rare JDBC types, vendor-specific functions, or highly specialized schemas are underrepresented. Databases in train/dev and test splits do not overlap, but latent similarity across domains is still possible. In addition, MyBatis-Spider is partially bootstrapped with GPT-4o, which we use as a construction assistant when synthesizing candidate mappings and requirements. All released instances are validated by deterministic execution and manually curated by domain experts, and our primary metric OSA compares the behavior of candidate and reference mappers rather than raw model suggestions; nonetheless, the use of an LLM in the construction process may still introduce mild stylistic biases in wording or mapping patterns that are not fully captured by our checks. For reproducibility, we document decoding settings, seeds, and executors; scripts reconstruct database state and apply the same strict XML parsing (with dynamic-tag checks) and object-alignment pipeline used in our experiments.

Threat: entity-schema correspondence in real codebases. AST aligns schema columns and entity fields positionally, which holds in MyBatis-Spider because entity classes are

generated from the schema with a canonical field order (schema-derived fields match the column order, while relationship/helper fields are appended afterward). In real-world projects, however, this assumption may be violated due to field reordering, inherited/embedded fields, computed/helper fields inserted in the middle, refactoring, or framework-specific annotations. When such deviations occur, purely positional alignment may mis-pair fields and columns and reduce the effectiveness of AST. A promising direction is to augment AST with lightweight matching signals (e.g., name similarity, type compatibility, keys/constraints, or learned matching) to recover correspondences without assuming identical ordering.

VII. CONCLUSION

This paper studies *Text-to-Mapper*, the task of generating executable MyBatis mapping files from natural-language requirements, as a bridge between academic work on code generation and ORM-centric industrial practice. We formalize Text-to-Mapper around an executable parse-build-execute-align pipeline that makes framework-level contracts and cross-artifact consistency between SQL results, `<resultMap>` declarations, and JavaBeans explicit. To support systematic study of this setting, we construct and release *MyBatis-Spider*, an executable benchmark that pairs natural-language descriptions with `mapper.xml` artifacts across 206 databases, together with an evaluation stack that drives candidate mappers end-to-end and uses OSA as an execution-level correctness signal.

Starting from mainstream Text-to-SQL prompting practices, we adapt task-description styles and schema renderings to the Text-to-Mapper setting and empirically examine their behavior. Failure-composition analysis shows that under these direct-transfer baselines, result-stage misalignment between SQL outputs, `<resultMaps>`, and JavaBeans is a primary source of errors in our benchmarked direct-transfer setting, and that non-execution metrics can diverge substantially from OSA. Motivated by these findings, we introduce AST, a prompt-level across-structure table that makes field-column-type correspondences explicit, and study it together with Across-Schema Alignment cues across both commercial and open-source LLM backbones. Experiments indicate that AST consistently reduces result-stage errors and achieves the highest observed OSA among the evaluated input designs, and that combining AST with lightweight LoRA enables mid-scale open-source models to narrow much of the gap to larger general-purpose models on this specialized benchmark, and in some settings match or slightly exceed them.

Future work includes broadening MyBatis-Spider beyond Spider-derived schemas and SQLite backends to cover additional application domains, ORM frameworks, and database engines while preserving executable, behavior-level ground truth, and scaling AST-style alignments to larger, highly normalized schemas, for example via sharded or more hierarchical variants. It is also natural to pair such input-level alignment with more advanced in-context learning strategies—such as explicit problem decomposition and few-shot or retrieval-based demonstrations—which are not evaluated in this study.

APPENDIX

A. Prompt Templates and Examples

We instantiate each requirement using two widely adopted prompting formats. **Text-style** uses free-form descriptions commonly seen in Text-to-SQL settings [49], resembling developer requests in documentation or issue tickets. **Alpaca-style** follows the instruction–input layout popularized in instruction tuning [50], which makes constraints explicit and easier to skim for long inputs.

For both templates, we keep the wording domain-agnostic and only minimally specialize it to Text-to-Mapper. Concretely, both templates instruct the model to: (i) generate an executable mapper.xml; (ii) use dynamic SQL tags (e.g., `<if>`, `<choose>`, `<foreach>`) when conditional logic or list expansion is required; (iii) project only the columns needed by the target entities; and (iv) prefer `resultMap` over `resultType` when nontrivial projection–property mapping is involved. Figure 12 shows example snippets for both formats.

Alpaca SFT Style

Below is an instruction that describes a task, paired with an input that provides further context. Write a response that appropriately completes the request.
Instruction:
 You are an expert Java developer specializing in the MyBatis ORM framework. Your task is to generate a complete, syntactically correct, and executable MyBatis Mapper XML file based on the specific requirement and the provided context.
Critical Guidelines:
 - ****Result Mapping**:** For complex queries with joins, you **MUST** use a `<resultMap>`. For simple cases, `<resultType>` is sufficient.
 - ****Dynamic SQL**:** Analyze the requirement for optional conditions and use dynamic SQL tags like `<if>` or `<where>` if needed.
 - ****Column Selection**:** The query must only select necessary columns.
 - ****Executability**:** Ensure the final code is functionally correct and executable.
The specific requirement is:
{NL requirement}
Input:
 The following context contains the relevant database schema and Java entity definitions:
{Schema Rendering}
Response:

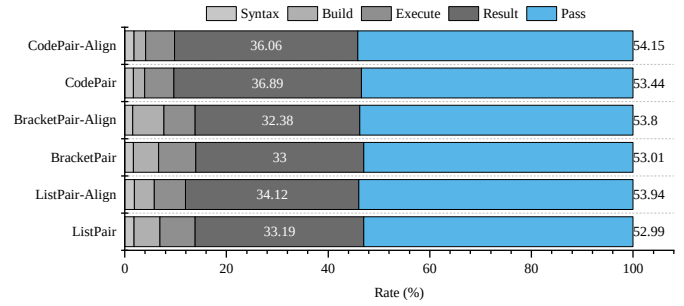
Text Representation Style

As an expert Java developer specializing in the MyBatis ORM framework, your task is to generate a complete, syntactically correct, and executable MyBatis Mapper XML file. Please carefully analyze the natural language requirement to determine if conditional logic is needed, and if so, utilize appropriate dynamic SQL tags like `<if>` or `<where>`. It is also critical that you only select the specific columns necessary to populate the target Java entity. For complex queries that involve joins or custom column-to-property mappings, you must create and use a `<resultMap>` to ensure correctness; otherwise, a simple `<resultType>` will suffice. Above all, the final generated XML must be functionally correct and executable in a real-world environment. Please complete this task based on the following requirement and the provided schema details.
Requirement:
{NL requirement}
Schema and Entity Information:
{Schema Rendering}

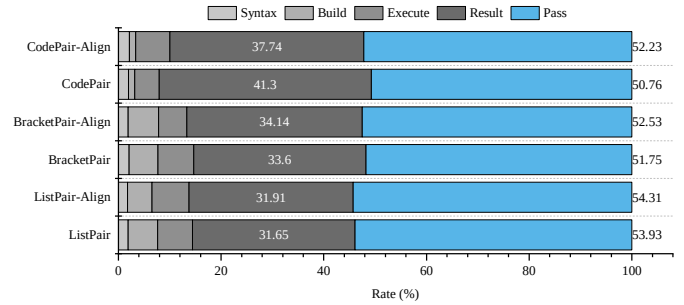
Fig. 12. Examples of task description formats: Alpaca SFT style and text representation style.

B. Per-template Breakdown for Direct Transfer

This appendix reports per-template results for the direct-transfer study (Section IV-B), complementing the main-text results aggregated as marginal means across templates. Specifically, we provide: (i) first-error composition under Text-style and Alpaca-style; and (ii) per-template executability gains introduced by -Align (ASA). Overall, the failure profile and the effect of lightweight alignment remain consistent across templates.



(a) Text Representation style



(b) Alpaca SFT style

Fig. 13. First-error composition on GPT-4o. Bars are 100% stacked by outcome label (Syntax/Build/Execute/Result/Pass), ordered by runtime stage. The rightmost segment (*Pass/OSA*) denotes runs that successfully complete all stages; the other segments indicate runs whose *first* failure occurs at each stage.

C. Template Sensitivity Summary

Table VII summarizes template sensitivity by reporting OSA under Text-style and Alpaca-style, together with Δ style, for each model (averaged over schema renderings and alignment variants). Template sensitivity is model-dependent; however, the main-text analyses focus on representation and alignment effects and therefore report marginal means across templates. We include the per-template summary here for reference and reproducibility.

TABLE VII
BLOCK ORDERING GAINS ON GPT-4O: Δ OSA = OSA(-ALIGN)–OSA(ORIGIN).

Rendering	Δ OSA (Text)	Δ OSA (Alpaca)
ListPair	+1.04	+0.85
BracketPair	+0.78	+0.78
CodePair	+0.71	+0.55

REFERENCES

- [1] P. Price, "Evaluation of spoken language systems: The atis domain," in *Speech and Natural Language: Proceedings of a Workshop Held at Hidden Valley, Pennsylvania, June 24-27, 1990*, 1990.
- [2] D. A. Dahl, M. Bates, M. K. Brown, W. M. Fisher, K. Hunnicke-Smith, D. S. Pallett, C. Pao, A. Rudnick, and E. Shriberg, "Expanding the scope of the atis task: The atis-3 corpus," in *Human Language Technology: Proceedings of a Workshop held at Plainsboro, New Jersey, March 8-11, 1994*, 1994.
- [3] S. Iyer, I. Konstas, A. Cheung, J. Krishnamurthy, and L. Zettlemoyer, "Learning a neural semantic parser from user feedback," *arXiv preprint arXiv:1704.08760*, 2017.
- [4] J. M. Zelle and R. J. Mooney, "Learning to parse database queries using inductive logic programming," in *Proceedings of the national conference on artificial intelligence*, 1996, pp. 1050–1055.
- [5] V. Zhong, C. Xiong, and R. Socher, "Seq2sql: Generating structured queries from natural language using reinforcement learning," *arXiv preprint arXiv:1709.00103*, 2017.
- [6] T. Yu, R. Zhang, K. Yang, M. Yasunaga, D. Wang, Z. Li, J. Ma, I. Li, Q. Yao, S. Roman *et al.*, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-sql task," *arXiv preprint arXiv:1809.08887*, 2018.
- [7] J. Li, B. Hui, G. Qu, J. Yang, B. Li, B. Li, B. Wang, B. Qin, R. Geng, N. Huo *et al.*, "Can llm already serve as a database interface? a big bench for large-scale database grounded text-to-sqls," *Advances in Neural Information Processing Systems*, vol. 36, 2024.
- [8] F. Lei, J. Chen, Y. Ye, R. Cao, D. Shin, H. Su, Z. Suo, H. Gao, W. Hu, P. Yin *et al.*, "Spider 2.0: Evaluating language models on real-world enterprise text-to-sql workflows," *arXiv preprint arXiv:2411.07763*, 2024.
- [9] A. Liu, X. Hu, L. Wen, and P. S. Yu, "A comprehensive evaluation of chatgpt's zero-shot text-to-sql capability," *arXiv preprint arXiv:2303.13547*, 2023.
- [10] N. Rajkumar, R. Li, and D. Bahdanau, "Evaluating the text-to-sql capabilities of large language models," *arXiv preprint arXiv:2204.00498*, 2022.
- [11] S. Chang and E. Fosler-Lussier, "How to prompt llms for text-to-sql: A study in zero-shot, single-domain, and cross-domain settings," *arXiv preprint arXiv:2305.11853*, 2023.
- [12] B. Wang, R. Shin, X. Liu, O. Polozov, and M. Richardson, "Rat-sql: Relation-aware schema encoding and linking for text-to-sql parsers," *arXiv preprint arXiv:1911.04942*, 2019.
- [13] J. Guo, Z. Zhan, Y. Gao, Y. Xiao, J.-G. Lou, T. Liu, and D. Zhang, "Towards complex text-to-sql in cross-domain database with intermediate representation," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Florence, Italy: Association for Computational Linguistics, 2019, pp. 4524–4535. [Online]. Available: <https://aclanthology.org/P19-1444/>
- [14] R. Cao, L. Chen, Z. Chen, Y. Zhao, S. Zhu, and K. Yu, "Lgesql: Line graph enhanced text-to-sql model with mixed local and non-local relations," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Online: Association for Computational Linguistics, 2021, pp. 2541–2555. [Online]. Available: <https://aclanthology.org/2021.acl-long.198/>
- [15] T. Yu, M. Yasunaga, K. Yang, R. Zhang, D. Wang, Z. Li, and D. Radev, "Syntaxsqlnet: Syntax tree networks for complex and cross-domain text-to-sql task," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Brussels, Belgium: Association for Computational Linguistics, 2018, pp. 1653–1663. [Online]. Available: <https://aclanthology.org/D18-1193/>
- [16] T. Scholak, N. Schucher, and D. Bahdanau, "Picard: Parsing incrementally for constrained auto-regressive decoding from language models," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, 2021, pp. 9895–9901. [Online]. Available: <https://aclanthology.org/2021.emnlp-main.779/>
- [17] C. Wang, K. Tatwawadi, M. Brockschmidt, P.-S. Huang, Y. Mao, O. Polozov, and R. Singh, "Robust text-to-sql generation with execution-guided decoding," 2018. [Online]. Available: <https://arxiv.org/abs/1807.03100>
- [18] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou, "Text-to-sql empowered by large language models: A benchmark evaluation," *arXiv preprint arXiv:2308.15363*, 2023.
- [19] S. Chang and E. Fosler-Lussier, "Selective demonstrations for cross-domain text-to-sql," in *Findings of the Association for Computational Linguistics: EMNLP 2023*. Singapore: Association for Computational Linguistics, 2023, pp. 14 174–14 189. [Online]. Available: <https://aclanthology.org/2023.findings-emnlp.944/>
- [20] J.-M. Favre, "Foundations of model (driven)(reverse) engineering: Models," in *Int. workshop dagstuhl*. Citeseer, 2004.
- [21] MyBatis Team, "Mybatis generator (mbg)," <https://mybatis.org/generator/>, 2024, accessed: 2025-06-10.
- [22] Baomidou Team, "Mybatis-plus: An enhanced toolkit of mybatis," <https://github.com/baomidou/mybatis-plus>, 2024, accessed: 2025-06-10.
- [23] A. Torres, R. d. M. Galante, and M. S. Pimenta, "A synergistic model-driven approach for persistence modeling with uml," *Journal of Systems and Software*, vol. 84, no. 6, pp. 942–957, 2011.
- [24] D. M. Groenewegen, Z. Hemel, L. C. L. Kats, and E. Visser, "Webdsl: A domain-specific language for dynamic web applications," in *Companion to the 23rd ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA '08 Companion)*. Nashville, TN, USA: ACM, 2008, pp. 779–780.
- [25] V. Schreibmann and P. Braun, "Model-driven development of restful apis," in *Proceedings of the 11th International Conference on Web Information Systems and Technologies (WEBIST)*, vol. 2. SciTePress, 2015, pp. 5–14.
- [26] M. Fowler, *Patterns of enterprise application architecture*. Addison-Wesley, 2012.
- [27] E. Evans, *Domain-driven design: tackling complexity in the heart of software*. Addison-Wesley Professional, 2004.
- [28] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: a method for automatic evaluation of machine translation," in *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, 2002, pp. 311–318.
- [29] C.-Y. Lin, "Rouge: A package for automatic evaluation of summaries," in *Text summarization branches out*, 2004, pp. 74–81.
- [30] G. Costa, G. Manco, R. Ortale, and A. Tagarelli, "A tree-based approach to clustering xml documents by structure," in *Knowledge Discovery in Databases: PKDD 2004*, ser. Lecture Notes in Computer Science, J.-F. Boulicaut, F. Esposito, F. Giannotti, and D. Pedreschi, Eds., vol. 3202. Berlin, Heidelberg: Springer, 2004, pp. 137–148.
- [31] J. Tekli, R. Chbeir, and K. Yétongnon, "An overview on XML similarity: Background, current trends and future directions," *Computer Science Review*, vol. 3, no. 3, pp. 151–173, 2009.
- [32] A. Nierman and H. V. Jagadish, "Evaluating structural similarity in XML documents," in *Proceedings of the 5th International Workshop on the Web and Databases (WebDB 2002)*, Madison, WI, USA, 2002, pp. 61–66.
- [33] N. Augsten, M. H. Böhlen, and J. Gamper, "Approximate matching of hierarchical data using pq-grams," in *Proceedings of the 31st International Conference on Very Large Data Bases (VLDB 2005)*. VLDB Endowment, 2005, pp. 301–312.
- [34] Y. Wang, D. J. DeWitt, and J. Cai, "X-diff: An effective change detection algorithm for XML documents," in *Proceedings of the 19th IEEE International Conference on Data Engineering (ICDE 2003)*, 2003, pp. 519–530.
- [35] S. Ren, D. Guo, S. Lu, L. Zhou, S. Liu, D. Tang, N. Sundaresan, M. Zhou, A. Blanco, and S. Ma, "Codebleu: a method for automatic evaluation of code synthesis," *arXiv preprint arXiv:2009.10297*, 2020.
- [36] T. Yu, R. Zhang, M. Yasunaga, Y. C. Tan, X. V. Lin, S. Li, H. Er, I. Li, B. Pang, T. Chen *et al.*, "Sparc: Cross-domain semantic parsing in context," *arXiv preprint arXiv:1906.02285*, 2019.
- [37] M. Pourreza and D. Rafiei, "Din-sql: Decomposed in-context learning of text-to-sql with self-correction," *Advances in Neural Information Processing Systems*, vol. 36, pp. 36 339–36 348, 2023.
- [38] C. Finegan-Dollak, J. K. Kummerfeld, L. Zhang, K. Ramanathan, S. Sadasivam, R. Zhang, and D. Radev, "Improving text-to-sql evaluation methodology," *arXiv preprint arXiv:1806.09029*, 2018.
- [39] N. Ding, Y. Qin, G. Yang, F. Wei, Z. Yang, Y. Su, S. Hu *et al.*, "Parameter-efficient fine-tuning of large-scale pre-trained language models," *Nature Machine Intelligence*, vol. 5, no. 3, pp. 220–235, 2023.
- [40] J. Bian, Y. Peng, L. Wang, Y. Huang, and J. Xu, "A survey on parameter-efficient fine-tuning for foundation models in federated learning," *arXiv preprint arXiv:2504.21099*, 2025.
- [41] M. Weyssow, X. Zhou, K. Kim, D. Lo, and H. Sahraoui, "Exploring parameter-efficient fine-tuning techniques for code generation with large language models," *arXiv preprint arXiv:2308.10462*, 2023.
- [42] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, W. Chen *et al.*, "Lora: Low-rank adaptation of large language models," *ICLR*, vol. 1, no. 2, p. 3, 2022.

- [43] “Hello gpt-4o,” <https://openai.com/index/hello-gpt-4o/>, May 2024.
- [44] T. GLM, A. Zeng, B. Xu, B. Wang, C. Zhang, D. Yin, D. Zhang, D. Rojas, G. Feng, H. Zhao *et al.*, “Chatglm: A family of large language models from glm-130b to glm-4 all tools,” *arXiv preprint arXiv:2406.12793*, 2024.
- [45] M. AI, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024. [Online]. Available: <https://arxiv.org/abs/2407.21783>
- [46] “Meta llama 3 github repository,” <https://github.com/meta-llama/llama3>, 2024.
- [47] DeepSeek-AI, “Deepseek-v3.2-exp: Boosting long-context efficiency with deepseek sparse attention,” 2025.
- [48] A. Liu *et al.*, “Deepseek-v3 technical report,” *arXiv preprint arXiv:2412.19437*, 2024. [Online]. Available: <https://arxiv.org/abs/2412.19437>
- [49] L. Nan, Y. Zhao, W. Zou, N. Ri, J. Tae, E. Zhang, A. Cohan, and D. Radev, “Enhancing text-to-sql capabilities of large language models: A study on prompt design strategies,” in *Findings of the Association for Computational Linguistics: EMNLP 2023*, 2023, pp. 14 935–14 956.
- [50] R. Taori, I. Gulati, T. Zhang, Y. Dubois, C. Guestrin, T. Hashimoto, and P. S. Liang, “Stanford alpaca: An instruction-following llama model,” *arXiv preprint arXiv:2304.05262*, 2023.